

UNIVERSIDAD
NACIONAL
DE COLOMBIA

Laboratory 4

Structural Architectural Patterns

Jeisson Andrés Vergara Vargas

Software Architecture

2026-II

---

1. Objective

The objective of this laboratory is to reuse the structure from the Lab 2 to design, deploy, and test a system architecture based on the **Broker** architectural pattern. We will implement a RabbitMQ message broker acting as the central connector between producers and consumers, and build components that publish and consume messages through queues.

2. Prerequisites

A computer with a Unix-based OS and Docker installed.

3. Components

Create a folder named `broker-pattern`.

3.1. Component 1 — Broker (RabbitMQ)

This component acts as the **central broker** of the architecture. It receives messages from producers and forwards them to consumers through a durable queue. No code is required; we use the official RabbitMQ Docker image with the management plugin enabled.

3.2. Component 2 — Producer (HTTP API)

This component exposes an HTTP API that allows clients to send messages via `POST /items`. Each message is published to the RabbitMQ queue.

Structure: `component-2/app`

a. app/main.py

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import os
import asyncio
import aio_pika
import json

app = FastAPI()

RABBIT_URL = os.environ.get("RABBIT_URL", "amqp://guest:guest@component-
1:5672/")
QUEUE_NAME = os.environ.get("QUEUE_NAME", "items_queue")

class Item(BaseModel):
    name: str
    description: str

async def get_connection():
    return await aio_pika.connect_robust(RABBIT_URL)

@app.post("/items")
async def publish_item(item: Item):
    try:
        connection = await get_connection()
        async with connection:
            channel = await connection.channel()
            await channel.default_exchange.publish(
                aio_pika.Message(
                    body=json.dumps(item.dict()).encode(),
                    delivery_mode=aio_pika.DeliveryMode.PERSISTENT
                ),
                routing_key=QUEUE_NAME,
            )
        return {"status": "published"}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

b. requirements.txt

```
fastapi
uvicorn
pydantic
aio-pika
python-dotenv
```

c. Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

3.3. Component 3 — Consumer (Worker)

This component acts as a **worker** that subscribes to the `items_queue`, processes each message, and stores it into a MongoDB database.

Structure: `component-3/worker`

a. `worker/worker.py`

```
import asyncio
import aio_pika
import json
import motor.motor_asyncio
import os

RABBIT_URL = os.environ.get("RABBIT_URL", "amqp://guest:guest@component-
1:5672/")
QUEUE_NAME = os.environ.get("QUEUE_NAME", "items_queue")
MONGO_URL = os.environ.get("MONGO_URL", "mongodb://component-4:27017")

mongo_client = motor.motor_asyncio.AsyncIOMotorClient(MONGO_URL)
db = mongo_client["broker_lab"]
collection = db["items"]

async def process_message(message: aio_pika.IncomingMessage):
    async with message.process():
        data = json.loads(message.body)
        await collection.insert_one(data)
        print(f"[x] Item stored: {data}")

async def main():
    connection = await aio_pika.connect_robust(RABBIT_URL)
    channel = await connection.channel()
    queue = await channel.declare_queue(QUEUE_NAME, durable=True)
    await queue.consume(process_message)
    print("[x] Waiting for messages...")
    return connection

if __name__ == "__main__":
    loop = asyncio.get_event_loop()
    connection = loop.run_until_complete(main())
    try:
        loop.run_forever()
```

```
    finally:
        loop.run_until_complete(connection.close())
```

b. `requirements.txt`

```
aio-pika
motor
python-dotenv
```

c. `Dockerfile`

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY worker ./worker
CMD ["python", "worker/worker.py"]
```

3.4. Component 4 — Database (MongoDB)

This component stores the processed messages from the consumer. It uses the official MongoDB Docker image.

4. Deployment

Create a `docker-compose.yml` file inside `broker-pattern`:

```
services:

  component-1:
    image: rabbitmq:3-management
    container_name: component-1
    ports:
      - "5672:5672"
      - "15672:15672"

  component-2:
    build: ./component-2
    container_name: component-2
    ports:
      - "8000:8000"
    environment:
      RABBIT_URL: amqp://guest:guest@component-1:5672/
      QUEUE_NAME: items_queue
    depends_on:
```

```
      - component-1

  component-3:
    build: ./component-3
    container_name: component-3
    environment:
      RABBIT_URL: amqp://guest:guest@component-1:5672/
      QUEUE_NAME: items_queue
      MONGO_URL: mongodb://component-4:27017
    depends_on:
      - component-1
      - component-4

  component-4:
    image: mongo
    container_name: component-4
    ports:
      - "27017:27017"
```

To build and run all containers:

```
docker-compose up --build
```

5. Connectors

In this architecture, connectors are represented by **message queues** in RabbitMQ. The producer (Component 2) publishes messages to the queue, and the consumer (Component 3) subscribes to the queue. RabbitMQ handles message routing and delivery, decoupling the two components.

Additionally, connectors exist through network communication between:

- Component 2 → Component 1 (AMQP protocol)
- Component 3 → Component 1 (AMQP protocol)
- Component 3 → Component 4 (db_connector)

6. Testing

6.1. Broker Dashboard

Access the RabbitMQ Management Dashboard at:
http://localhost:15672
Username: guest
Password: guest

You can monitor queues, connections, and consumers here.

6.2. Producer API

Send a message through the HTTP producer:

```
curl -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "SwArch", "description": "2026-II"}'
```

You should see `{ "status": "published" }`.

6.3. Consumer Logs

In the consumer container logs, you should see something like:

```
[x] Item stored: {'name': 'SwArch', 'description': '2026-II'}
```

6.4. Database Verification

Access the MongoDB container:

```
docker exec -it component-4 mongosh
```

Then check the inserted documents:

```
use broker_lab
db.items.find().pretty()
```

You should see the published messages stored in the database.

7. Delivery

7.1. Deliverable

- Component-and-Connector (C&C) View.
- Deployment View.

7.2. Delivery Deadline

Tuesday, October 27, 2026 (in class).